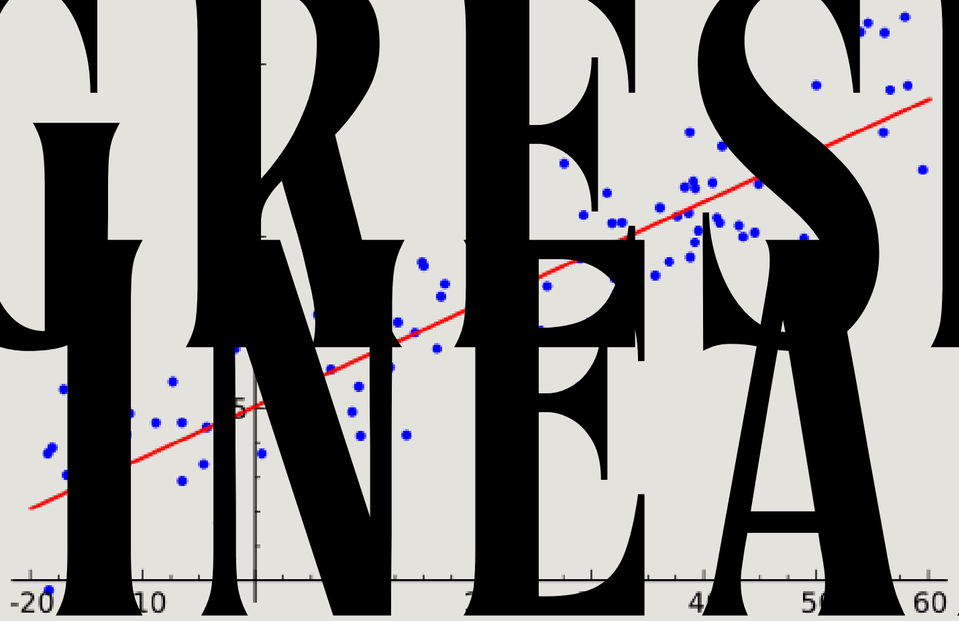
REGRESIÓN LINEAL



EN JULIA (GLM-TEMP_W-PLUTO)

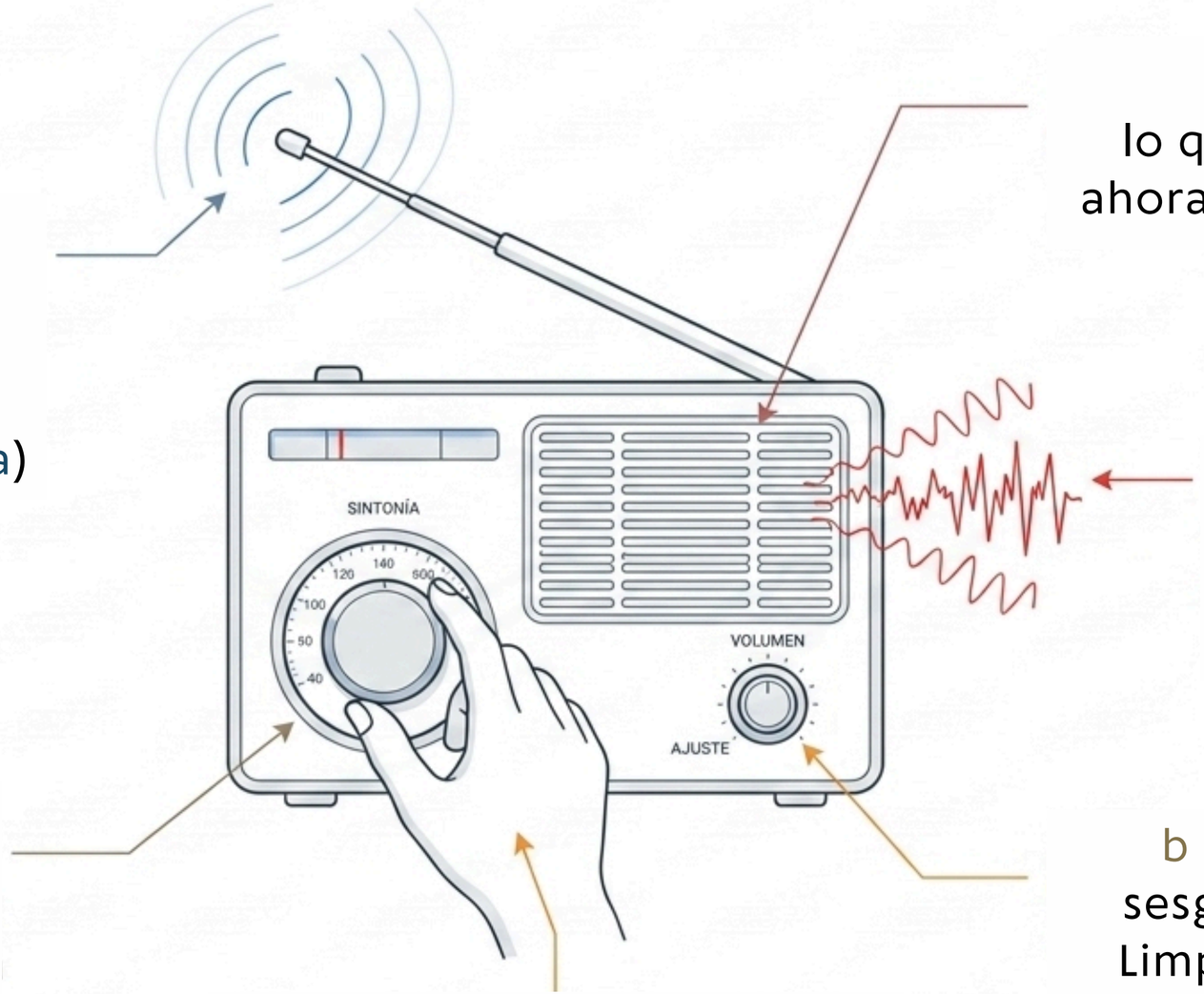
ÍNDICE

- Analogía de regresión lineal con el radio.....(1)
- modelo f(x)= w*x + b.....(2)
- Función de costo.....(3)
- Derivadas parciales.....(4)
- Gradiente descendente.....(5)
- Tablero de control.....(6)
- Geometría y espacio.....(7)
- Caso 1.....(8)
- Caso 2.....(9)
- Caso 3.....(10)
- Caso 4.....(11)
- Conclusión.....(12)
- Referencias.....(13)

1

X y Y (ondas y canción)
x: ondas de radio invisibles (datos de entrada)
y: canción original (meta)

F(x) (la bocina)
lo que sale de la radio ahora mismo (predicción)



J (La estática)
Qué tan lejos está el ruido de la canción

w (perilla principal)
peso / pendiente.
Define el procesamiento central de la señal.

b (Ajuste fino)
sesgo/intercepto.
Limpia el zumbido eléctrico de fondo

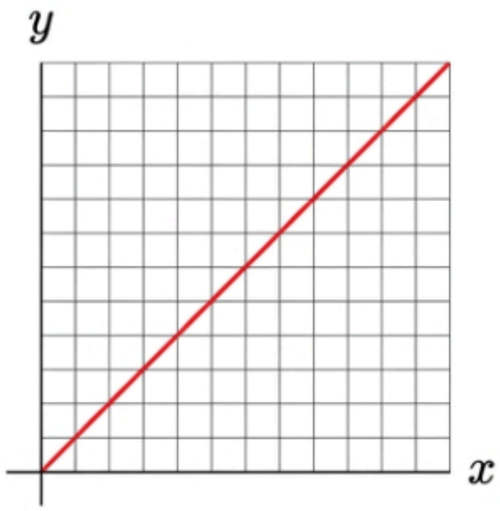
alfa(tasa de aprendizaje)
la brusquedad o fuerza de la mano al girar las perillas

EL MODELO

EN JULIA

$$f_{(w,b)}(x) = wx + b$$

w es la inclinación b es el corte en el eje




```
function modelo (w, b, x)
    return w * x + b
end
```

En julia, la traducción de la fórmula matemática al código fuente es literal y directa.

FUNCIÓN DE COSTO

$$J(w,b) = \frac{1}{2m} \sum \left(f_{(w,b)}(x^{(i)}) - y^{(i)}\right)^2$$

- El cuadrado: penaliza errores grandes severamente y evita que positivos y negativos se cancelen.
- El truco de 1/2: Al derivar la función, el 2 del exponente baja y se cancela. Es únicamente como si fuera limpieza matemática

EN JULIA

```julia
function funcion_costo(w, b, x_datos, y_datos)
    m = length(x_datos)   # cuantos puntos tenemos en el dataset
    suma = sum((modelo(w, b, x_datos[i]) - y_datos[i])^2 for i in 1:m)
    return suma / (2.0 * m)   # el 1/2 es un truco: al derivar el 2 del
end
```

- **NOTA: EL BUCLE EXPLÍCITO SUM() NO PENALIZA EL RENDIMIENTO AQUI, GRACIAS AL COMPILADOR JIT ESPECIALIZADO DE JULIA.**

4 DERIVADAS PARCIALES

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{m} \sum (\hat{\mathbf{y}} - \mathbf{y}) \cdot \mathbf{x}$$

$$\frac{\partial J}{\partial \mathbf{b}} = \frac{1}{m} \sum (\hat{\mathbf{y}} - \mathbf{y})$$

- Las derivadas son la flecha matemática como la voz de tu conciencia que susurra en qué dirección exacta debes girar cada perilla para que la estática disminuya.

EN JULIA

```julia
function dj_dw(w, b, x_datos, y_datos)
    m = length(x_datos)
    suma = sum((modelo(w, b, x_datos[i]) - y_datos[i]) * x_datos[i] for i in 1:m)
    return suma / m
end
```

```julia
function dj_db(w, b, x_datos, y_datos)
    m = length(x_datos)
    suma = sum(modelo(w, b, x_datos[i]) - y_datos[i] for i in 1:m)   # sin x porque
    return suma / m
end
```

- **NOTA: EL BUCLE EXPLÍCITO SUM() NO PENALIZA EL RENDIMIENTO AQUI, GRACIAS AL COMPILADOR JIT ESPECIALIZADO DE JULIA.**

GRADIENTE DESCENDENTE (GIRANDO PERILLAS AL UNÍSONO)

IMPLEMENTACIÓN PRÁCTICA

Error: Actualizar W y usar el nuevo W ára calcular la derivada de b.

Solución correcta (swap):

```
temp_w = w - alfa * grad_w
temp_b = b - alfa * grad_b
w = temp_w
b = temp_b
```

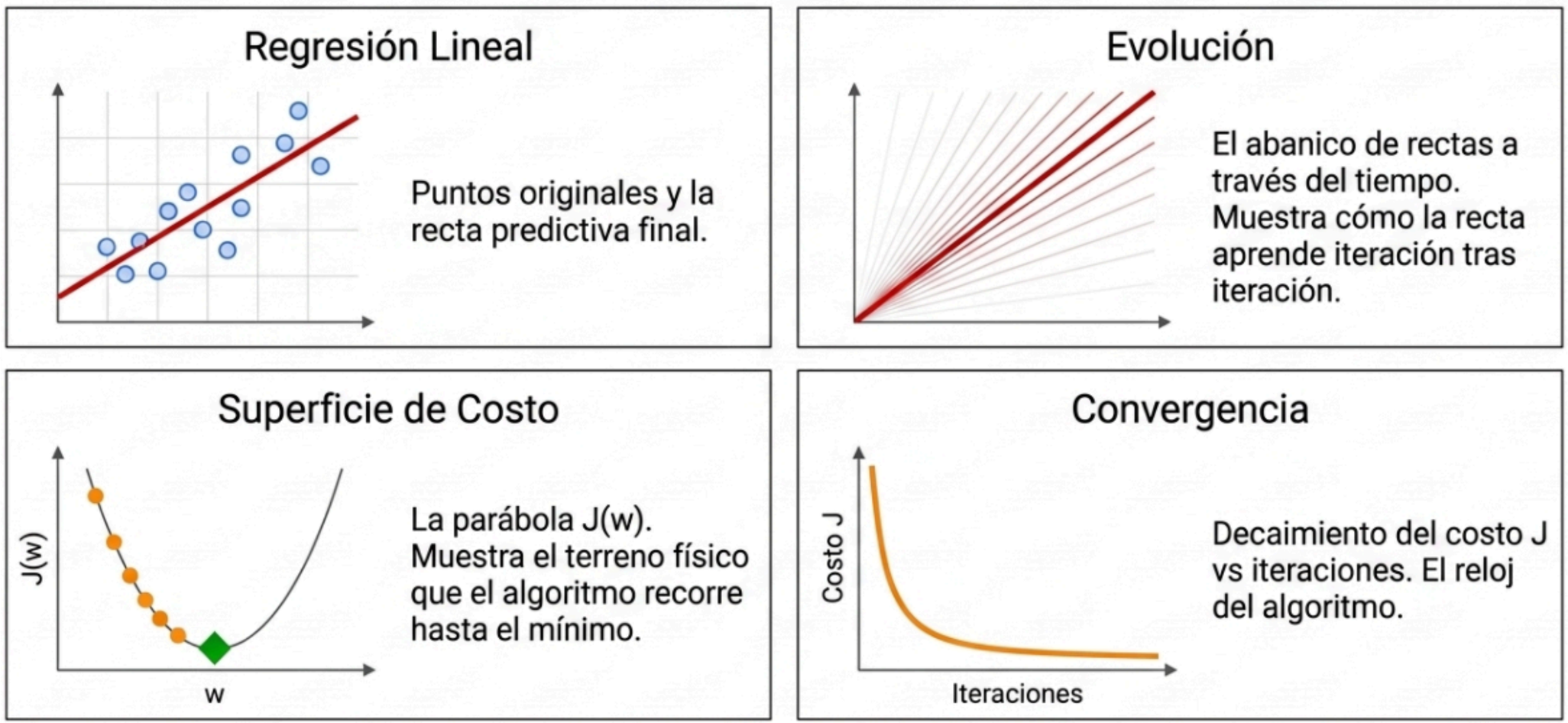
Se calcula variables temporales primero con un solo paso unificado hacia el óptimo

REGLA TEÓRICA

$$\text{Actualización: } w \leftarrow w - \alpha \frac{\partial J}{\partial w}$$

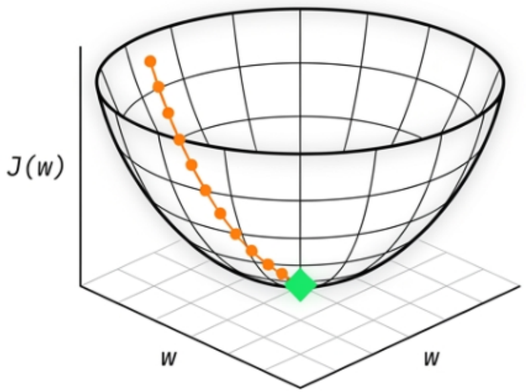
El descenso de gradiente exige que las actualizaciones de los parámetros sean estrictamente simultáneas

TABLERO DE CONTROL: LECTURA DE GRÁFICAS



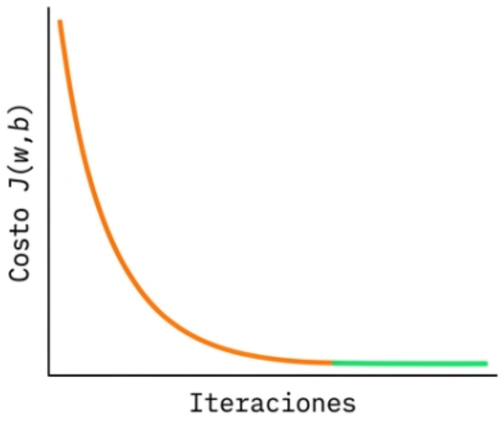
GEOMETRÍA Y ESPACIO

PARÁBOLA



- Gráfica J(w) con respecto de w
- Representa el área matemática
- Muestra por qué funciona el algoritmo con superficie convexa.

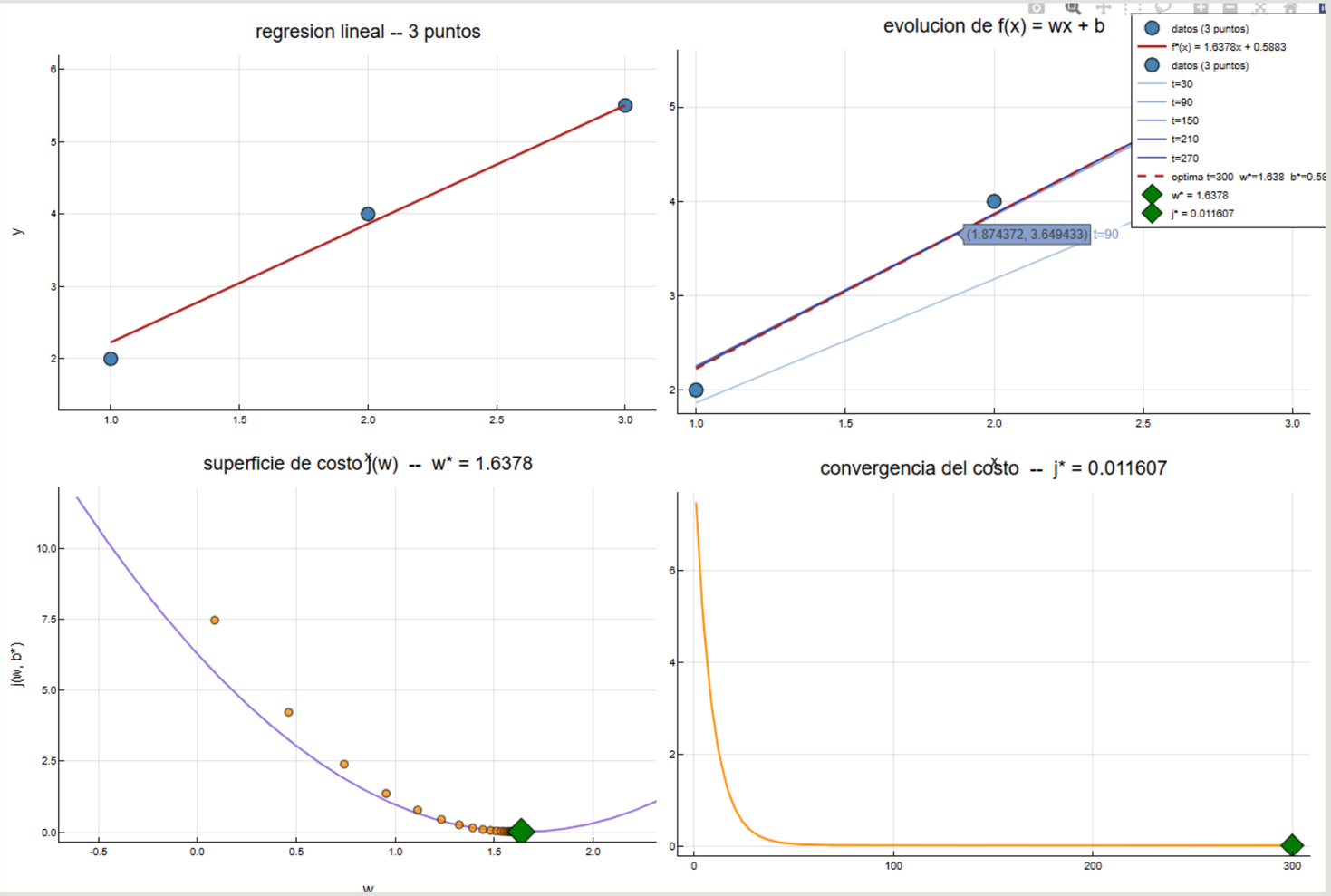
CONVERGENCIA



- Gráfica J(w,b) con respecto de iteraciones
- Muestra que tan rápido desaparece la estática
- Indica cuando el modelo ha terminado de aprender

CASO 1: CONVERGENCIA IDEAL

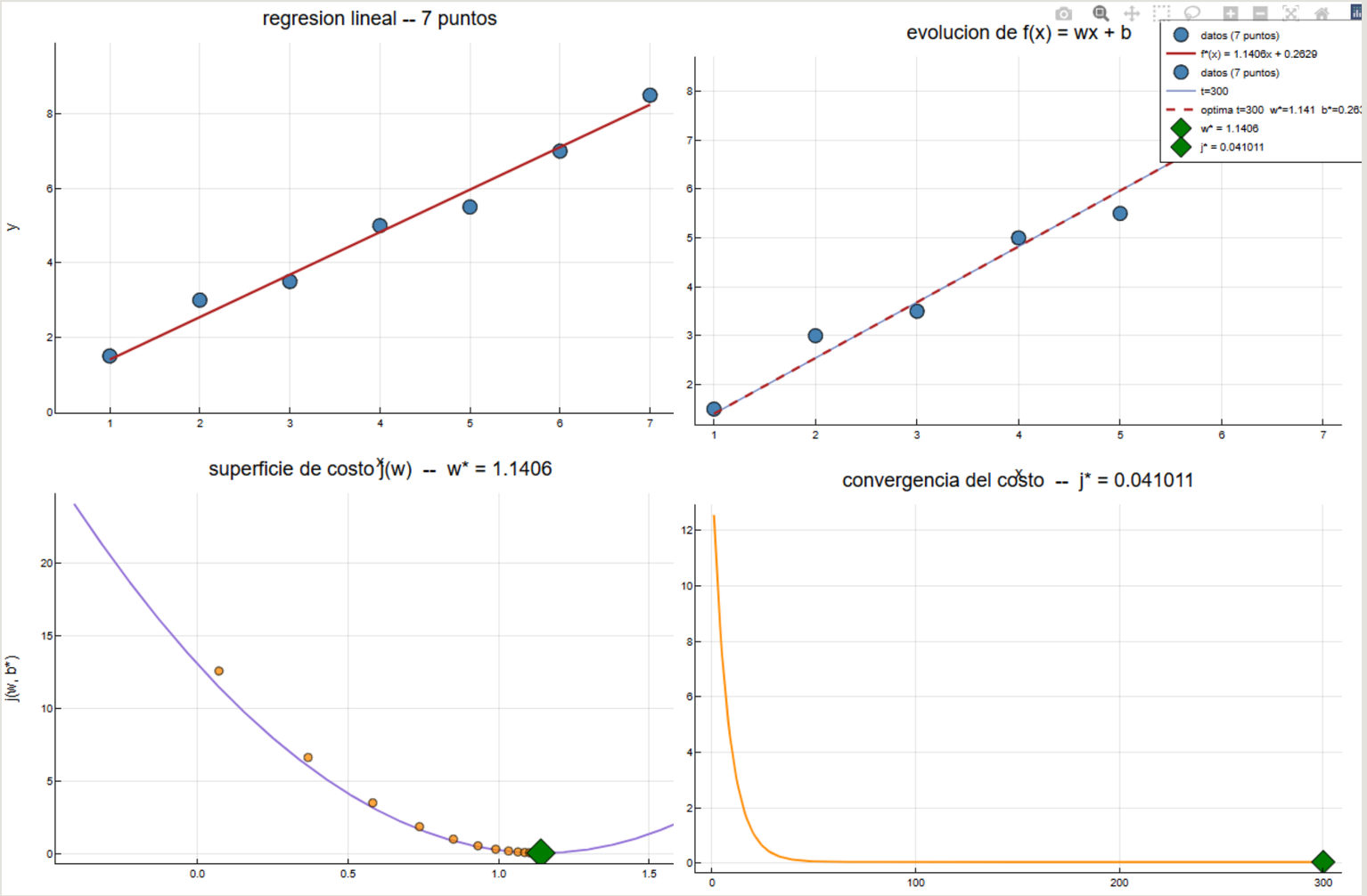
conjunto de datos: 3 puntos learning rate (alfa): 0.01 numero de iteraciones: 300



- **Observación:** la curva decae exponencialmente en 50 iteraciones y se aplana
- **Geomtría:** los puntos naranjas aterrizan casi óptimamente al diamante verde.
- Girar la perilla con pulso firme, desaparece el ruido.

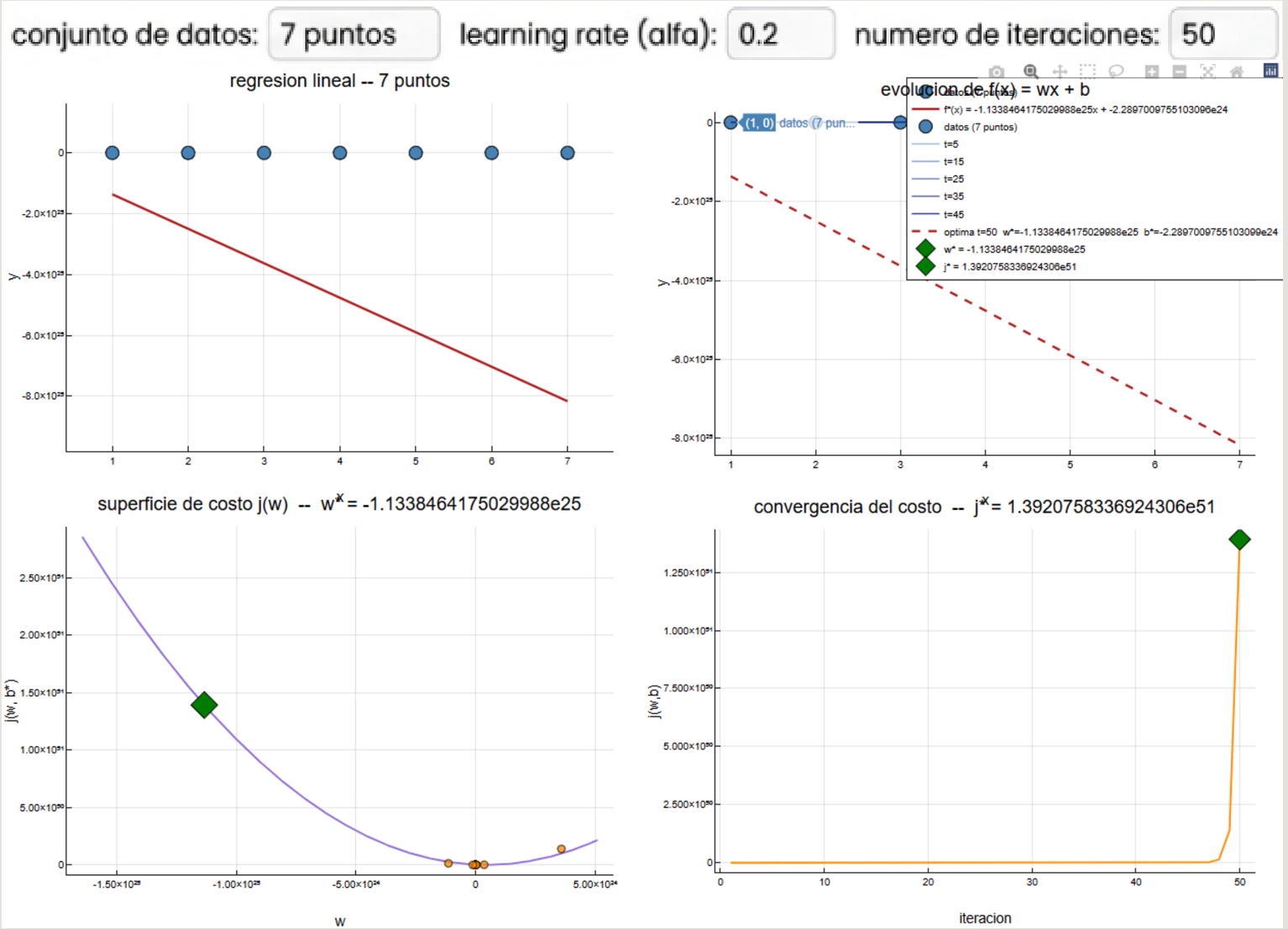
CASO 2: AJUSTE FINO

conjunto de datos: 7 puntos | learning rate (alfa): 0.003 | numero de iteraciones: 300



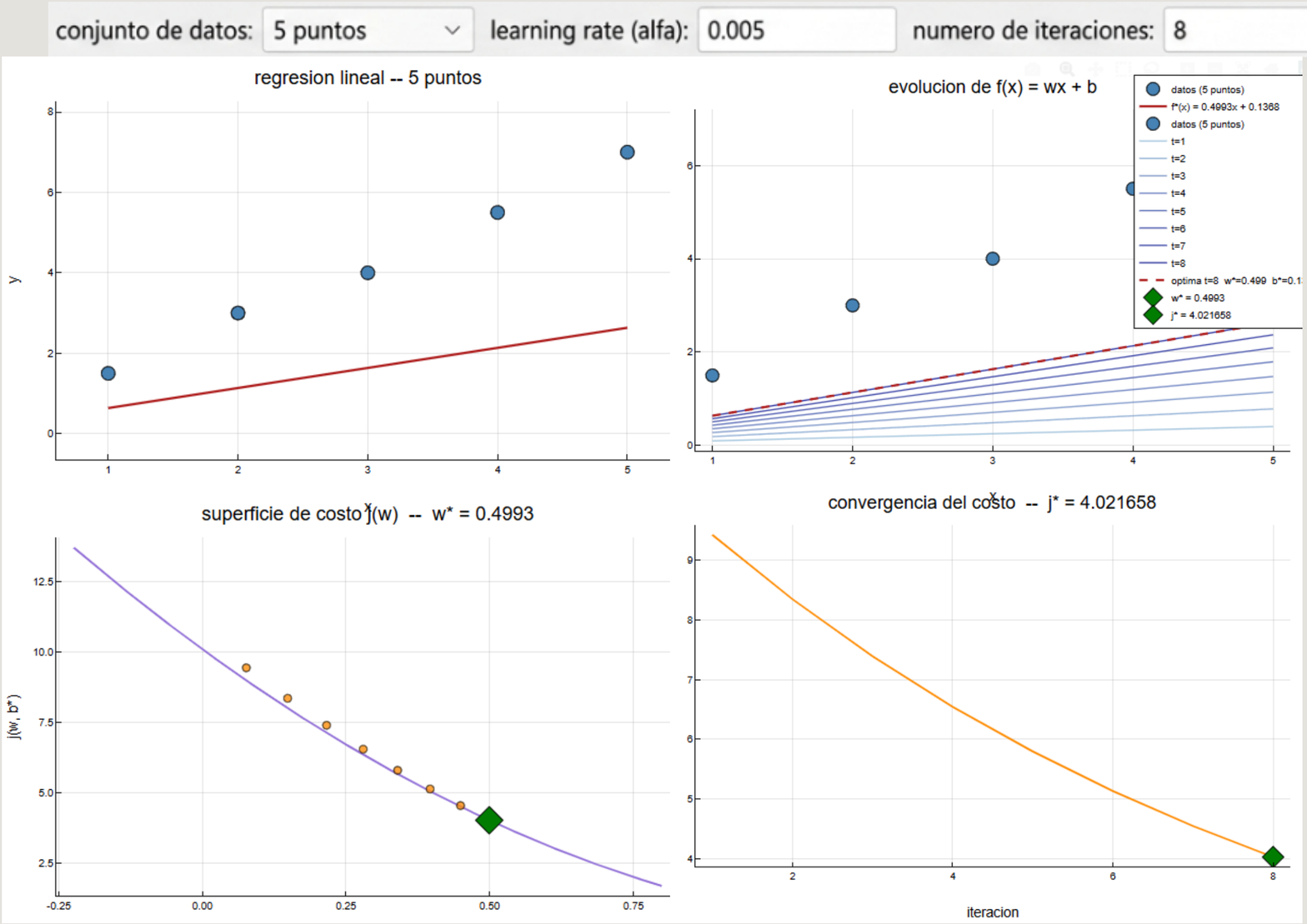
- **Reto:** al tener valores de x más grandes, los gradientes mutpliciados hacen pasos más gigantes.
- **Solución:** reducir alfa a 0.003 para compensar la escala geométrica.
- **Resultado:** pasos más pequeños y densos pero con aterrizaje más adecuado

CASO 3: ERROR O DIVERGENCIA



- **Fallo matemático:** alpha sobrepasa el límite teórico de estabilidad. Cada salto rebota más lejos del mínimo.
- **Colapso:** el error explota a magnitudes de $10^{51.}$ la curva se va para abajo
- Queda como detalle que la mano es tan brusca que pasa la estación de largo, rebota violentamente al otro lado y no encuentra la frecuencia.

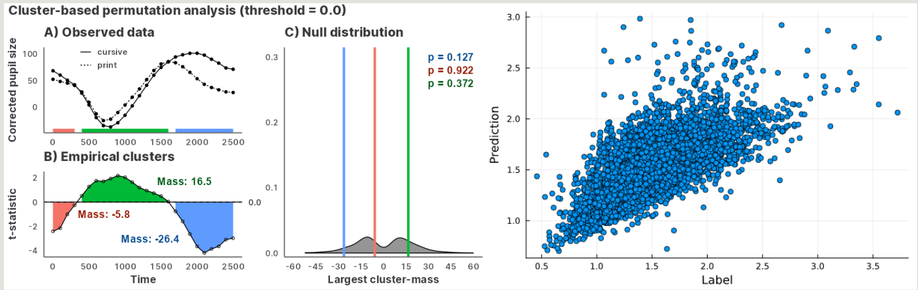
CASO 4: FALTA DE PACIENCIA



- **Síntomas visuales:** la curva de convergencia sigue cayendo sin aplanarse.
- **Geometría:** el diamante verde queda atascado a la mitad de la ladera matemática con 0.50
- Apagar la máquina por imapciencia justo cuando la estática se empezaba a disipar.

CONCLUSIÓN

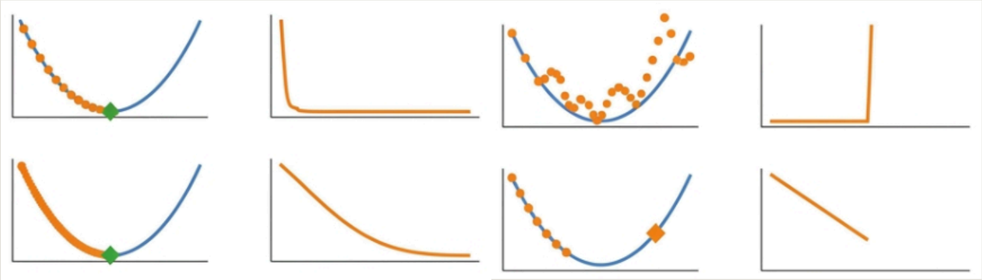
LIBRERÍAS O BLIBLIOTECAS



using GLM\lm@formula(y - x), df)

Eficiente y robusto. Pero signfica aceptar ciegamente la tolerancia, estabilidad numérica y el algoritmo interno validados por alguien o por algo más.

CÓDIGO MANUAL



temp_w = w - alfa * grad_w

Frágil y expuesto a colapsos matemáticos como NaN, pero completamente transparente. Cada variable es una decisión consciente.

El gradiente descendente es literalmente rodar hacia abajo por una superficie matemática. El esqueleto que sostiene el aprendizaje automático moderno